

Esercitazione Server Linux.

Obiettivo

In questo laboratorio, userai la riga di comando Linux per identificare i server in esecuzione su un dato computer

- Parte 1 Server
- Parte 2 Usare Telnet per Testare i Servizi TCP

Iniziamo avviando la VM CyberOps ed entriamo come utente analyst e password cyberops.



Apriamo il cmd e controlliamo tutti i processi attivi con il comando “sudo ps -elf”. Il comando ps-elf mostra tutti i processi del sistema, inclusi quelli appartenenti ad altri utenti, appartenenti a root, relativi ai servizi di sistema, gestiti da demoni che contengono informazioni sensibili (es. parametri, path, chiavi, token). Linux non permette a un utente normale di vedere tutti i dettagli dei processi degli altri utenti per questo utilizziamo il comando con sudo.

```
[analyst@secOps ~]$ sudo ps -elf
[sudo] password for analyst:
Sorry, try again.
[sudo] password for analyst:
Sorry, try again.
[sudo] password for analyst:
F S UID          PID     PPID    C  PRI   NI   ADDR  SZ  WCHAN  STIME TTY          TIME CMD
4 S root           1         0    0   80    0   5500 do_epo 04:16 ?    00:00:00 /sbin/init
1 S root           2         0    0   80    0      0 kthrea 04:16 ?    00:00:00 [kthreadd]
1 S root           3         2    0   80    0      0 kthrea 04:16 ?    00:00:00 [pool_workqueue_release]
1 I root           4         2    0   60  -20    0 rescue 04:16 ?    00:00:00 [kworker/R-rcu_gp]
1 I root           5         2    0   60  -20    0 rescue 04:16 ?    00:00:00 [kworker/R-sync_wq]
1 I root           6         2    0   60  -20    0 rescue 04:16 ?    00:00:00 [kworker/R-kvfree_rcu_reclaim]
1 I root           7         2    0   60  -20    0 rescue 04:16 ?    00:00:00 [kworker/R-slub_flushwq]
1 I root           8         2    0   60  -20    0 rescue 04:16 ?    00:00:00 [kworker/R-netns]
1 I root          12         2    0   80    0      0 worker 04:16 ?    00:00:00 [kworker/u8:0-events_unbound]
1 I root          13         2    0   80    0      0 worker 04:16 ?    00:00:00 [kworker/u8:1-ipv6_addrconf]
1 I root          14         2    0   60  -20    0 rescue 04:16 ?    00:00:00 [kworker/R-mm_percpu_wq]
1 S root          15         2    0   80    0      0 smpboo 04:16 ?    00:00:00 [ksoftirqd/0]
1 I root          16         2    0   58    0      0 rcu_gp 04:16 ?    00:00:00 [rcu_preempt]
1 S root          17         2    0   58    0      0 rcu_bo 04:16 ?    00:00:00 [rcub/0]
1 S root          18         2    0   80    0      0 kthrea 04:16 ?    00:00:00 [rcu_exp_par_gp_kthread_worker/0]
1 S root          19         2    0   80    0      0 kthrea 04:16 ?    00:00:00 [rcu_exp_gp_kthread_worker]
1 S root          20         2    0  -40    0      0 smpboo 04:16 ?    00:00:00 [migration/0]
1 S root          21         2    0    9    0      0 smpboo 04:16 ?    00:00:00 [idle_inject/0]
1 S root          22         2    0   80    0      0 smpboo 04:16 ?    00:00:00 [cpuhp/0]
1 S root          23         2    0   80    0      0 smpboo 04:16 ?    00:00:00 [cpuhp/1]
1 S root          24         2    0    9    0      0 smpboo 04:16 ?    00:00:00 [idle_inject/1]
1 S root          25         2    0  -40    0      0 smpboo 04:16 ?    00:00:00 [migration/1]
1 S root          26         2    0   80    0      0 smpboo 04:16 ?    00:00:00 [ksoftirqd/1]
1 I root          28         2    0   60  -20    0 worker 04:16 ?    00:00:00 [kworker/1:0H-events_highpri]
1 I root          29         2    0   80    0      0 worker 04:16 ?    00:00:00 [kworker/u9:0-events_unbound]
5 S root          31         2    0   80    0      0 devtmp 04:16 ?    00:00:00 [kdevtmpfs]
1 I root          32         2    0   60  -20    0 rescue 04:16 ?    00:00:00 [kworker/R-inet_frag_wq]
1 I root          33         2    0   80    0      0 rcu_ta 04:16 ?    00:00:00 [rcu_tasks_kthread]
1 I root          34         2    0   80    0      0 rcu_ta 04:16 ?    00:00:00 [rcu_tasks_rude_kthread]
```

Avviamo il server nginx con il comando sudo per avere privilegi elevati.

```
[analyst@secOps ~]$ sudo systemctl start nginx
[analyst@secOps ~]$ sudo systemctl status nginx
• nginx.service - nginx web server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: active (running) since Tue 2025-12-09 04:32:11 EST; 11s ago
 Invocation: 32a80850a6ea47a08d016fbb8e10b46c
   Process: 832 ExecStart=/usr/bin/nginx (code=exited, status=0/SUCCESS)
  Main PID: 833 (nginx)
    Tasks: 2 (limit: 2332)
   Memory: 4.3M (peak: 4.7M)
      CPU: 18ms
   CGroup: /system.slice/nginx.service
           └─833 "nginx: master process /usr/bin/nginx"
             └─834 "nginx: worker process"

Dec 09 04:32:11 secOps systemd[1]: Starting nginx web server...
Dec 09 04:32:11 secOps systemd[1]: Started nginx web server.
[analyst@secOps ~]$
```

E ora ricontrolliamo i processi attivi questa volta utilizzando l'opzione -ejH.

```
[analyst@secOps ~]$ sudo ps -ejH
```

PID	PGID	SID	TTY	TIME	CMD
2	0	0	?	00:00:00	kthreadd
3	0	0	?	00:00:00	pool_workqueue_release
4	0	0	?	00:00:00	kworker/R-rcu_gp
5	0	0	?	00:00:00	kworker/R-sync_wq
6	0	0	?	00:00:00	kworker/R-kvfree_rcu_reclaim
7	0	0	?	00:00:00	kworker/R-slub_flushwq
8	0	0	?	00:00:00	kworker/R-netns
12	0	0	?	00:00:00	kworker/u8:0-events_unbound
13	0	0	?	00:00:00	kworker/u8:1-ipv6_addrconf
14	0	0	?	00:00:00	kworker/R-mm_percpu_wq
15	0	0	?	00:00:00	ksoftirqd/0
16	0	0	?	00:00:00	rcu_preempt
17	0	0	?	00:00:00	rcub/0
18	0	0	?	00:00:00	rcu_exp_par_gp_kthread_worker/0
19	0	0	?	00:00:00	rcu_exp_gp_kthread_worker
20	0	0	?	00:00:00	migration/0
21	0	0	?	00:00:00	idle_inject/0
22	0	0	?	00:00:00	cpuhp/0
23	0	0	?	00:00:00	cpuhp/1
24	0	0	?	00:00:00	idle_inject/1
25	0	0	?	00:00:00	migration/1
26	0	0	?	00:00:00	ksoftirqd/1
28	0	0	?	00:00:00	kworker/1:0H-events_highpri
29	0	0	?	00:00:00	kworker/u9:0-events_unbound
31	0	0	?	00:00:00	kdevtmpfs
32	0	0	?	00:00:00	kworker/R-inet_frag_wq
33	0	0	?	00:00:00	rcu_tasks_kthread
34	0	0	?	00:00:00	rcu_tasks_rude_kthread
35	0	0	?	00:00:00	rcu_tasks_trace_kthread
36	0	0	?	00:00:00	kauditd
37	0	0	?	00:00:00	khungtaskd
38	0	0	?	00:00:00	oom_reaper
39	0	0	?	00:00:00	kworker/u10:1-kvfree_rcu_reclaim
40	0	0	?	00:00:00	kworker/u10:2-flush-8:0
41	0	0	?	00:00:00	kworker/R-writeback
47	0	0	?	00:00:00	kcompactd

Infondo osserviamo che ci sono appunto i processi di nginx :

Master process (root)

- Ha PID = PGID = SID = 833
- È il processo principale, avviato come root
- Gestisce:
 - lettura configurazione
 - apertura porte privilegiate (80/443)
 - creazione dei worker

✓ Worker process (www-data / nginx user)

- Ha PID 834
- PPID = 833, quindi è figlio del master
- Gestisce le richieste HTTP vere e proprie
- Gira con privilegi ridotti per sicurezza

```
747      510      510 ?        00:00:00 x11vnc-terminal
758      758      758 pts/0    00:00:00 bash
858      858      758 pts/0    00:00:00 sudo
860      860      860 pts/1    00:00:00 sudo
861      861      860 pts/1    00:00:00 ps
833      833      833 ?        00:00:00 nginx
834      833      833 ?        00:00:00 nginx
[analyst@secOps ~]$
```

Usiamo netstat per verificare il server di rete in esecuzione e vediamo nginx: master p. Utilizziamo le opzioni -tunap: -t mostra le connessioni TCP, -u mostra le connessioni UDP, -n fa visualizzare indirizzi e porte in forma numerica senza tentare la risoluzione dei nomi, -a mostra tutte le connessioni e le porte in ascolto, -p visualizza il PID e il nome del processo associato a ciascuna connessione. Per netstat l'ordine delle opzioni non è importante. Puoi scriverle in qualunque sequenza, come -tunap, -naptu o -pantu, e il comando funzionerà allo stesso modo perché netstat interpreta ogni opzione singolarmente senza dare importanza all'ordine in cui viene inserita.

Dall'output mostrato si vede chiaramente la riga relativa alla porta 80, il protocollo di Livello 4 è TCP, lo stato della connessione è LISTEN e il PID del processo è 833. Poiché la porta 80 TCP è tradizionalmente utilizzata per il traffico HTTP e il processo in ascolto è nginx, si può dedurre che è in esecuzione un server web HTTP.

```
[analyst@secOps ~]$ sudo netstat -tunap
[sudo] password for analyst:
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6633            0.0.0.0:*               LISTEN      370/python3.9
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      471/vsftpd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      395/sshd: /usr/bin/
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      833/nginx: master p
tcp6       0      0 :::22                   :::*                    LISTEN      395/sshd: /usr/bin/
udp        0      0 10.215.77.140:68        0.0.0.0:*               295/systemd-network
[analyst@secOps ~]$
```

Dal comando netstat vediamo che il processo di nginx è 833 quindi usiamo ps -elf con | grep 833 per listare solo i processi con quel PID. Si può concludere che il processo 833 è nginx perché nella colonna del comando compare chiaramente “nginx: master process /usr/bin/nginx”, quindi il nome del processo indica che si tratta del processo principale di nginx. Nginx è un web server e reverse proxy molto utilizzato per servire pagine web, gestire traffico HTTP e fare da bilanciatore o proxy. La seconda riga mostra che il processo 834 appartiene all'utente http e ha come processo genitore il PID 833. Questo significa che il master process di nginx ha creato uno o più worker process, che vengono eseguiti con un utente con meno privilegi. Questo comportamento è normale: il processo principale gira come root per aprire porte privilegiate, mentre i worker girano come utenti non privilegiati per motivi di sicurezza. L'ultima riga mostra “grep 833” semplicemente perché il comando ps è stato filtrato con grep, quindi grep stesso appare nei risultati in quanto contiene la stringa 833 che si sta cercando.

```
[analyst@secOps ~]$ sudo ps -elf | grep 833
1 S root      833      1 0 80  0 - 3723 sigsus 04:32 ?        00:00:00 nginx: master process /usr/bin/nginx
5 S http     834      833 0 80  0 - 3827 do_ep0 04:32 ?        00:00:00 nginx: worker process
0 S analyst  958      758 0 80  0 - 1615 anon_p 04:55 pts/0    00:00:00 grep 833
[analyst@secOps ~]$
```


Ora utilizziamo telnet per verificare il funzionamento del web server e verificarne l'autenticità. Quindi ci colleghiamo a local host nella porta 80 ovvero la porta nginx. Inviando un messaggio casuale per vedere quale sarà la risposta. L'errore è stato inviato come pagina web perché ci siamo collegati alla porta 80, che è la porta del servizio HTTP gestito da nginx. Quando un server web riceve una richiesta non valida, come nel nostro caso dove abbiamo scritto semplicemente "ciao" invece di un vero comando HTTP, il server risponde comunque secondo le regole del protocollo HTTP. Questo significa che deve restituire una risposta formata da uno status code, delle intestazioni e un corpo in formato HTML. Per questo motivo anche l'errore viene mostrato come una pagina web, perché nginx parla sempre il linguaggio HTTP quando ascolta sulla porta 80.

```
[analyst@secOps ~]$ telnet 127.0.0.1 80
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
ciao
HTTP/1.1 400 Bad Request
Server: nginx/1.28.0
Date: Tue, 09 Dec 2025 10:01:40 GMT
Content-Type: text/html
Content-Length: 157
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.28.0</center>
</body>
</html>
Connection closed by foreign host.
[analyst@secOps ~]$
```

Sebbene il server abbia segnalato un errore e terminato la connessione, siamo stati in grado di imparare molto.

Abbiamo imparato che:

1. L'nginx con PID 833 è di fatto un server web.
2. La versione di nginx è 1.16.1.
3. Lo stack di rete della nostra VM CyberOps Workstation è completamente funzionante fino al Livello 7.

Sempre con l'uso di telnet proviamo a connetterci alla porta 68 e vediamo che non riesce a connettersi in quanto non c'è nessun servizio che gira in quella porta.

```
[analyst@secOps ~]$ telnet 127.0.0.1 68
Trying 127.0.0.1...
telnet: Unable to connect to remote host: Connection refused
[analyst@secOps ~]$
```

Domande di Riflessione 1. Quali sono i vantaggi dell'uso di netstat?

Netstat è utile perché permette di vedere quali porte sono in ascolto, quali processi le utilizzano e quali connessioni sono attive, aiutando a capire lo stato della rete e a individuare problemi o attività sospette

2. Quali sono i vantaggi dell'uso di Telnet? È sicuro?

Telnet è vantaggioso perché permette di testare rapidamente connessioni a porte specifiche e vedere se un servizio risponde, ed è molto semplice da usare per il troubleshooting. Tuttavia non è sicuro, perché trasmette tutto in chiaro, incluse eventuali credenziali, quindi non dovrebbe essere usato per accedere a sistemi reali ma solo per test locali o ambienti di laboratorio.

